

Python Programming

Assignment 1

Sanika Mahind
Roll Number: MS2517

Part A: Python Programming Concepts

1 Variables

What is it? A variable is a label attached to a value stored in memory, so the program can refer to that value later using the label instead of the raw data. Python decides the type of a variable automatically based on whatever is assigned to it.

```
age = 22  
name = "Sanika"
```

Why do we need it? Without variables we would have to repeat literal values everywhere in the code. Variables let us store a value once, reuse it, and update it as the program runs.

Where and How is it used? Variables appear in essentially every Python program – to hold numbers, text, boolean flags, user input, and results of computations.

```
name = "Sanika"  
marks = 91  
passed = True  
print(name)  
print(marks)  
print(passed)
```

2 Basic Data Types

A data type describes the kind of value a variable is holding. Python's basic built-in types include Integer, Float, Boolean, and String.

2.1 Integer

What is it? An `int` is a whole number, with no fractional part. It may be positive, negative, or zero.

```
age = 22  
temperature = -3  
count = 0
```

Why do we need it? We need integers to represent and compute with counts and whole quantities.

Where and How is it used? Ages, roll numbers, number of items, loop counters.

```
students = 45
marks = 91
```

2.2 Float

What is it? A float is a number that contains a decimal point.

```
height = 5.4
temperature = 36.7
```

Why do we need it? Many real-world quantities are not whole numbers, so we need a type that can store fractional values.

Where and How is it used? Percentages, prices, measurements, averages.

```
percentage = 91.5
price = 249.50
```

2.3 Boolean

What is it? A bool can only hold one of two values, **True** or **False**.

```
is_student = True
is_absent = False
```

Why do we need it? Programs frequently need to represent whether a condition holds or not, and booleans capture exactly that.

Where and How is it used? Conditions, decision-making, and flags.

```
age = 19
is_adult = age >= 18
print(is_adult)
```

2.4 String

What is it? A str is a sequence of characters, used to represent text.

```
name = "Sanika"
course = "Scientific Computing"
```

Why do we need it? Programs need to work with textual data such as names and messages, and strings are Python's way of representing that text.

Where and How is it used? Names, messages, addresses, user input.

```
name = "Sanika"
message = "Welcome to Python"
print(message)
```

3 Arithmetic Operators

What is it? Arithmetic operators perform mathematical computations on numeric values: `+`, `-`, `*`, `/`, `//`, `%`, and `**`.

Why do we need it? They let a program carry out calculations instead of the programmer computing results by hand.

Where and How is it used? Computing totals, averages, and other numeric results.

```
a = 12
b = 5
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a % b)
```

4 Comparison Operators

What is it? Comparison operators compare two values and produce `True` or `False`: `==`, `!=`, `>`, `<`, `>=`, `<=`.

Why do we need it? They allow a program to test relationships between values so it can make decisions.

Where and How is it used? Conditional statements and loop conditions.

```
age = 19
print(age >= 18)
print(age < 18)
```

5 Assignment Operators

What is it? Assignment operators store or update a value in a variable. The basic one is `=`; Python also has shorthand operators such as `+=`, `-=`, `*=`, and `/=`.

Why do we need it? They let us set a variable's value and update it concisely, without rewriting the variable name on both sides.

Where and How is it used? Whenever a stored value needs to change.

```
x = 10
x += 5
print(x)
x -= 3
print(x)
```

6 Type Conversion

What is it? Type conversion is changing a value from one data type into another, using functions such as `int()`, `float()`, `str()`, and `bool()`.

Why do we need it? Data does not always arrive in the type we need – for example, `input()` always returns a string – so we convert it to perform the correct operation.

Where and How is it used? Handling user input and performing numeric operations on it.

```
age = "19"
age = int(age)
print(age + 1)
```

7 Input and Output

7.1 input()

What is it? input() reads a line of text typed by the user while the program is running.

Why do we need it? It lets a program become interactive by accepting information from the user rather than having it hard-coded.

Where and How is it used? Collecting details such as a name, age, or marks.

```
name = input("Enter your name: ")
```

7.2 print()

What is it? print() displays text or values on the screen.

Why do we need it? It is how a program communicates results and messages back to the user.

Where and How is it used? Displaying any kind of output.

```
name = "Sanika"
print("Hello", name)
```

8 Strings

A string is a sequence of characters. Python provides ways to create strings, access individual characters, extract portions of them, and combine them.

8.1 String Creation

Strings can be written using either single or double quotes.

```
name = "Python"
message = 'Hello'
```

8.2 Indexing

Indexing accesses one character of a string by its position. Python indexing starts at 0, and negative indices count from the end.

```
text = "Python"
print(text[0])
print(text[-1])
```

Here `text[0]` is the first character and `text[-1]` is the last character.

8.3 Slicing

Slicing extracts a sub-part of a string using a start, stop, and optional step.

```
text = "Python Programming"
print(text[0:6])
print(text[::-1])
```

`text[0:6]` gives Python, and `text[::-1]` reverses the string by stepping backwards through it.

8.4 String Operations

Strings can be joined with `+` (concatenation) and repeated with `*`.

```
first = "Python"
second = "Programming"
print(first + " " + second)
print("=" * 20)
```

9 Lists

What is it? A list is an ordered, mutable collection that can hold several values in one variable.

Why do we need it? Lists let us group related values together and work with them as a single unit instead of many separate variables.

Where and How is it used? Storing marks, names, or any collection of related data.

9.1 Creating Lists

```
numbers = [10, 20, 30]
fruits = ["Apple", "Banana", "Mango"]
```

9.2 Indexing

```
fruits = ["Apple", "Banana", "Mango"]
print(fruits[0])
print(fruits[-1])
```

9.3 Slicing

```
numbers = [10, 20, 30, 40, 50]
print(numbers[1:4])
```

9.4 Updating Elements

Because lists are mutable, an element can be reassigned after the list is created.

```
numbers = [10, 20, 30]
numbers[1] = 200
print(numbers)
```

9.5 Nested Lists

A list can itself contain lists as elements.

```
matrix = [  
    [1, 2],  
    [3, 4]  
]  
print(matrix[0][1])
```

9.6 Basic List Methods

- `append()` – adds an element at the end
- `insert()` – inserts an element at a given position
- `remove()` – removes a given element
- `pop()` – removes and returns an element
- `sort()` – sorts the list
- `reverse()` – reverses the list
- `len()` – returns the number of elements

```
numbers = [30, 10, 20]  
numbers.append(40)  
numbers.insert(1, 15)  
numbers.remove(10)  
numbers.sort()  
numbers.reverse()  
print(numbers)  
print(len(numbers))
```

10 GitHub Repository

All programs in this assignment were executed in Jupyter Notebook and pushed to GitHub.

Repository Link: <https://github.com/SanikaMahind>